

P01 Semantic Identity and the Subject Registry

Executable research capsule

PBUI research prototype

2026-08-05

Contents

1 P01 - Semantic Identity and the Subject Registry	1
1.1 Research question	1
1.2 Why this subsystem exists	2
1.3 Executable model	2
1.3.1 Semantic sorts	2
1.3.2 Alternate representations	3
1.3.3 Alias graph	3
1.3.4 Comparison evidence	3
1.3.5 Canonicalization	4
1.3.6 Revisioned snapshots	4
1.4 Formal model	4
1.5 Relationship to unification theory	5
1.6 Tests and experiments	6
1.7 Results established by the prototype	6
1.7.1 By construction	6
1.7.2 By executable tests	6
1.7.3 Not yet proved	6
1.8 Deliberate limitations	6
1.9 Integration seam	7
1.10 Research extensions	7

1 P01 - Semantic Identity and the Subject Registry

1.1 Research question

How should a presentation-based UI decide that two independently produced values denote the same application subject, without confusing domain identity with JavaScript reference equality, structural equality, React reconciliation keys, or screen occurrence identity?

This capsule builds a small executable answer based on:

- nominal semantic sorts;
- sort-scoped stable keys;
- alternate representation declarations;

- explicit, evidence-bearing alias edges;
- reversible equivalence classes derived from those edges;
- revisioned current snapshots that remain separate from identity.

1.2 Why this subsystem exists

The current PBUI reference shape carries a semantic type and a value:

```
{ type: "field", value: { docId: "doc-readings", name: "temperature" } }
```

That is enough to render and dispatch local actions, but it leaves several questions unspecified:

- Are two newly allocated field objects with equal coordinates the same field?
- Is a project card presenting a project object the same subject as a token presenting its project ID?
- Can a document ID and a user ID both equal to "1" collide?
- What happens when imported identifiers are discovered to alias canonical identifiers?
- Can an alias later be revoked without losing its provenance?
- Does an immutable update change identity?
- How is stale captured data detected?

P01 makes those decisions explicit.

1.3 Executable model

1.3.1 Semantic sorts

A sort defines a domain of identity:

```
interface SubjectSort<A> {
  name: string;
  key(value: A): string;
  revision?(value: A): number;
  fingerprint?(value: A): string;
}
```

Example:

```
const Project = {
  name: "project",
  key: project => project.id,
  revision: project => project.revision,
};
```

The semantic reference is:

```
interface SubjectRef {
  sort: string;
  key: string;
}
```

The sort is part of the identity domain. Therefore:

```
project:1 != document:1
```

unless an application deliberately defines a common super-domain outside this registry.

1.3.2 Alternate representations

A representation explains how another displayed value identifies a subject of an existing sort:

```
const ProjectId = {  
  name: "project-id-token",  
  sort: Project,  
  key: id => id,  
};
```

Both calls produce the same semantic coordinate:

```
registry.ref(Project, project)  
registry.refFromRepresentation(ProjectId, project.id)
```

No conversion of the visual representation into an entity snapshot is required merely to establish identity.

1.3.3 Alias graph

Sometimes two different keys are discovered to denote one subject:

```
registry.addAlias(  
  { sort: "project", key: "legacy-17" },  
  { sort: "project", key: "project-17" },  
  { reason: "workspace import", authority: "migration-v4" },  
);
```

Aliases are stored as explicit edges. The relation used by same is the reflexive, symmetric, transitive closure of those edges.

The implementation intentionally does not persist only a destructive union-find partition. Retaining generating edges provides:

- provenance;
- deterministic export;
- alias removal;
- recomputation after corrections;
- a clear distinction between asserted equations and their equivalence closure.

For the small registry prototype, breadth-first graph traversal is adequate. A production runtime can cache connected components or use dynamic connectivity while retaining the edge graph as the source of truth.

1.3.4 Comparison evidence

compare(left, right) returns either:

```
{ same: true, evidence: { kind: "same-key", ... } }
```

or:

```
{
  same: true,
  evidence: {
    kind: "alias-path",
    path: [edge1, edge2],
  },
}
```

or a negative reason such as different-sort or disconnected.

This evidence is useful for inspectors, migration diagnostics, audit trails, and future proof-carrying adapters. The Boolean convenience method `same` is derived from the evidence-producing operation.

1.3.5 Canonicalization

`canonical(ref)` returns the lexicographically least key in the current equivalence class. This policy is deterministic under alias insertion order.

The canonical representative is a runtime normalization policy, not the definition of identity. Consumers that need persistent application-preferred keys should define a stronger authority or ranking policy rather than depend on lexicographic order.

1.3.6 Revisioned snapshots

Identity and current data are separate:

```
registry.register(Project, {
  id: "project-17",
  title: "Compiler",
  revision: 4,
});
```

The registry rejects:

- a lower revision for the same semantic reference;
- a different fingerprint at the same revision.

A greater revision updates the current snapshot without changing the subject reference.

This demonstrates an important design rule:

A semantic reference should remain stable while the value currently describing that subject evolves.

1.4 Formal model

For each sort S , let K_S be its set of keys. Alias declarations generate a relation:

$$E_S \subseteq K_S \times K_S.$$

The semantic sameness relation is the least equivalence relation containing E_S :

$$\sim_S = \text{EqClosure}(E_S).$$

A subject reference is a pair:

$$(S, k), \quad k \in K_S.$$

Sameness is defined by:

$$(S, k) \approx (T, j) \iff S = T \wedge k \sim_S j.$$

The implementation tests reflexivity, symmetry, and transitivity over generated finite graphs.

A snapshot store is a partial map:

$$\text{snapshot} : \sum_S K_S \rightarrow (\mathbb{N} \times \text{Value}_S).$$

Its update rule requires monotone revisions. This order is independent of the alias equivalence relation.

1.5 Relationship to unification theory

Unification provides a useful contrast but is not the implementation of subject identity.

A first-order unifier answers whether variables can be substituted so that symbolic terms become equal. P01 receives ground domain coordinates and application assertions. It does not infer that two projects are identical because their fields can be unified, nor does it search for a substitution between arbitrary entity values.

The additional Baader-Snyder chapter is therefore used negatively here:

- **unifiability is not domain identity;**
- a domain identity theory must be declared rather than accidentally inherited from object structure;
- if future aliases are inferred modulo an equational theory, that theory and its decidability properties must be explicit;
- the term-DAG equivalence-class techniques motivate possible optimization, but do not replace the domain contract.

See the repository-level [docs/UNIFICATION-NOTES.md](#).

1.6 Tests and experiments

The capsule currently contains nine executable tests covering:

1. one subject through an entity object and an ID token;
2. rejection of accidental structural equality;
3. sort-domain collision isolation;
4. reflexive, symmetric, transitive aliases with path evidence;
5. deterministic canonicalization under different edge orders;
6. removal of an alias and resulting component split;
7. stale and conflicting revision rejection;
8. serialization and restoration of alias topology;
9. equivalence laws over a generated finite graph.

Run:

```
npm test
npm run demo
```

1.7 Results established by the prototype

1.7.1 By construction

- Cross-sort aliases are rejected.
- Alternate representations can only target declared sorts.
- Snapshot identity is the pair (sort, key) and does not depend on object allocation.
- The durable alias state retains its generating evidence.

1.7.2 By executable tests

- same behaves as an equivalence relation on the tested finite graph.
- Canonicalization is independent of alias insertion order for the selected policy.
- Removing an edge can split an equivalence class.
- Snapshot revision checks reject stale or conflicting values.

1.7.3 Not yet proved

- completeness and minimality of the path evidence;
- asymptotic guarantees for a production dynamic-connectivity implementation;
- correctness of a future union-find cache with deletion;
- correspondence between TypeScript and a mechanized quotient model;
- safe behavior under concurrent distributed alias edits.

1.8 Deliberate limitations

1. Keys are strings. A production API should accept codecs and nominal key types.
2. The default fingerprint is a stable best-effort serialization, not a cryptographic content hash.
3. Lexicographic canonicalization is a neutral experiment policy, not a product recommendation.

4. Alias imports allocate new local alias IDs. Semantic topology, not those IDs, is the portable invariant.
5. The registry does not resolve authorization for alias creation.
6. The registry does not infer aliases from labels, values, or unification.
7. Snapshots are in-memory and single-process.

1.9 Integration seam

P01 exports `SubjectRef`. P02 should attach that reference to occurrences. P03 should query records keyed by it. P04 should encode it into ground facts through an adapter.

A composed implementation must decide whether selector evidence stores the original reference, the current canonical reference, or both. The recommended approach is both:

```
{
  presented: SubjectRef,
  canonicalAtRevision: SubjectRef,
  identityEvidence: SameSubjectEvidence,
}
```

That preserves what the user acted on while allowing canonical state access.

1.10 Research extensions

- formalize the alias graph as a setoid and its quotient in Lean;
- compare explicit graph closure, union-find caching, and fully dynamic connectivity;
- add authority and revocation semantics to alias evidence;
- model identity migration across schema versions;
- test identity under fork, merge, import, and anonymization;
- investigate groupoid-style invertible representation witnesses;
- add distributed alias-edge CRDT semantics without collapsing value reconciliation into topology.